

CAMPUS EXCHANGE

Project Report

**Buy • Sell • Exchange with Students on Your
Campus**

**Submitted By
Sujal Rai**

Under the Guidance of: Rajshri Thete ma'am

Project Report prepared in the academic format of the reference Online News Portal report.

Index

Sr.no	Topic	Page no
1	Title of Project	1
2	Acknowledgement	3
3	Abstract	4
4	Introduction	5
5	Objective	6
6	System Analysis	7
7	System Design	22
8	Screenshots	32
9	Coding	39
10	Testing	46
11	Report	47
12	Future Scope	48
13	Conclusion	48
14	Bibliography	49
15	References	49

Acknowledgement

The project “Campus Exchange” is the project work carried out by

Name: Sujal Rai

Under the Guidance.

I am thankful to my project guide and mentors for guiding me throughout the development of the Campus Exchange web application. Their suggestions and valuable technical feedback helped in completing the project and its related documentation.

I am also thankful to my family members, friends, and everyone who supported me during the planning, coding, testing, debugging, and deployment stages of the project.

The project combines a student marketplace, messaging, buying/deal workflow, exchange workflow, favorites, nearby seller discovery, and profile management into a single campus-focused platform.

Abstract

Campus Exchange is a student-focused web application designed to make it easier for students to buy, sell, and exchange useful products within their campus community. Instead of depending on informal communication channels, the system provides a structured digital marketplace where students can create listings, browse available products, save favorites, contact sellers, and complete buying or exchange workflows. The system replaces scattered informal exchanges with a structured marketplace where product information, communication, preferences, and transaction records can be maintained in one place.

The application provides product categories such as Books, Notes, Electronics, Lab Equipment, Stationery, and Other items. A listing contains a title, description, price, condition, category, listing type, status, seller information, images, and optional geographic coordinates. Product images can be handled through Cloudinary, while browser geolocation supports nearby-seller discovery. Listings also contain structured information such as title, description, price, condition, listing type, availability status, and optional location coordinates, helping students compare items before contacting sellers.

The frontend is implemented with React, React Router, Axios, and Vite. The backend follows a layered FastAPI architecture using routers, services, Pydantic schemas, SQLAlchemy, JWT authentication, and a MySQL database. Messaging, deals, exchanges, favorites, and reviews provide additional interaction between students. React handles the component-based interface, React Router controls page navigation, Axios manages HTTP communication, and Vite supports the frontend development and build process.

The system is intended to be simple, responsive, secure, and practical for a college environment. It reduces the difficulty of finding relevant campus items and creates a centralized workflow for student-to-student transactions.

1. Introduction

In today's age of Information Communication and Technology, web applications have become an important part of everyday activities. Students frequently need books, notes, calculators, electronics, laboratory equipment, stationery, and other academic resources, while other students may already own those resources and no longer need them.

Campus Exchange addresses this situation by creating a digital marketplace specifically for students. The platform allows students to publish items, discover products from other students, communicate with sellers, save useful listings, and choose between buying and exchanging. Students can publish items, search and filter available resources, save useful listings, communicate with sellers, and choose either a purchase or exchange workflow.

The system also considers the physical nature of campus transactions. Optional latitude and longitude values allow the application to support nearby-seller discovery, helping students find available products around them.

The application follows a client-server architecture. The React frontend communicates with a FastAPI backend through HTTP APIs. The backend validates requests, applies business rules, and stores persistent information in MySQL through SQLAlchemy. The frontend focuses on presentation and user interaction, the FastAPI backend handles validation and business rules, and MySQL provides persistent relational storage.

2. Objective

The objectives of this project are as follows:

- To develop a web application for buying, selling, and exchanging items among students.
- To provide a centralized marketplace for academic and campus-related products.
- To allow students to create, view, manage, and remove their own listings.
- To provide category, condition, price, and search-based browsing.
- To provide direct messaging between students about listings.
- To support normal purchases through a structured Deal workflow.
- To support student-to-student product exchanges through an Exchange workflow.
- To provide favorites so students can save listings for later.
- To support nearby-seller discovery using browser location coordinates.
- To maintain transaction and listing status consistently in the database.
- To implement authentication and protected operations using JWT tokens.
- To provide a scalable foundation for future campus marketplace features.

3. System Analysis

3.1 PROBLEM DEFINITION

Students often buy and sell used academic materials through informal groups, messaging applications, or word of mouth. These approaches make it difficult to search for specific items, compare listings, identify nearby sellers, preserve transaction history, or coordinate exchanges.

Campus Exchange solves these problems by providing one organized platform. Listings are stored with structured attributes, users can communicate around a listing, and transaction workflows are represented by dedicated database records.

3.2 Preliminary Investigation

Purpose: The purpose of the system is to make student-to-student marketplace activity easier, more searchable, and more organized.

The application brings together listing management, search, favorites, messaging, buying, exchanging, nearby discovery, and user profiles in a single interface.

System Analysis — Benefits and Proposed System

Benefits

- Centralized marketplace for campus items.
- Faster discovery through categories and filters.
- Transparent listing information such as condition and price.
- Direct communication between buyer and seller.
- Favorites for saving products.
- Structured Deal and Exchange status management.
- Nearby seller discovery using optional location data.
- Reduced dependence on informal groups and manual coordination.

Proposed System

The proposed system provides a structured digital workflow in which a student registers, logs in, browses or creates listings, communicates with other students, and chooses a purchase or exchange action. The backend validates requests and persists the resulting state in MySQL. After finding an item, the student can save it, contact the owner, initiate a deal, or propose an exchange. This creates a complete workflow from discovery to transaction.

The system is designed to keep listing ownership and transaction roles explicit, so users cannot simply modify another user's listing or create a purchase against their own listing.

3.3 Feasibility Study

The feasibility study evaluates whether Campus Exchange is practical, achievable, and beneficial with the available technologies and resources.

Technical Feasibility

- React, Vite, React Router, and Axios provide a modern browser-based frontend.
- FastAPI provides a lightweight and high-performance API backend.
- SQLAlchemy provides ORM-based database access.
- MySQL provides persistent relational storage.
- JWT authentication protects authenticated operations.
- Cloudinary can be used for hosted product images.
- Browser geolocation supports latitude and longitude capture.

Economic Feasibility

- The primary software technologies are open-source.
- A student project can be developed and tested with modest hardware.
- Cloud services can be used selectively and scaled as needed.

Operational Feasibility

- The interface is designed around familiar marketplace actions.
- The system separates browsing, listings, messages, deals, exchanges, and profile functions.
- Students can use the application through a standard web browser.

3.4 Software Requirement Specification (SRS)

System Overview

Campus Exchange consists of a student-facing frontend and an API-driven backend. The system is organized around the following major modules: The backend exposes feature-specific APIs while the frontend consumes those APIs and presents the results through dedicated marketplace pages.

- Authentication Module — registration, login, JWT token handling, and protected requests.
- Listing Module — create, browse, view, update, and remove product listings.
- Messaging Module — student-to-student conversations linked to listings.
- Favorites Module — save and remove listings from the user's favorites.
- Deal Module — normal purchase workflow with buyer/seller confirmation.
- Exchange Module — offer one listing in return for another listing.
- Nearby Sellers Module — discover listings using geographic coordinates.
- Profile Module — display and update student account information.
- Review Module — collect feedback after completed deals.

The development workflow follows requirement analysis, design, implementation, testing, deployment, and maintenance. The implementation is modular so that individual features can be refined without redesigning the entire system. The modular approach also makes debugging easier because authentication, listing, communication, and transaction responsibilities can be checked separately before end-to-end testing.

Software & Hardware Requirements

Software Requirements

- Operating System: Windows, Linux, or macOS.
- Frontend: React, React Router, Axios, Vite, CSS.
- Backend: Python, FastAPI, SQLAlchemy, Pydantic.
- Authentication: JWT with password hashing.
- Database: MySQL.
- Media: Cloudinary for hosted images.
- Browser: Modern Chrome, Edge, Firefox, or equivalent.

Hardware Requirements

- Processor: Intel i3/i5 or equivalent.
- RAM: Minimum 4 GB; 8 GB recommended for development.
- Storage: At least 20 GB free for development tools and project files.
- Network: Internet access for hosted APIs, media, and deployment.

3.5 Functional Requirements

- Users shall be able to register and log in.
- Authenticated users shall be able to create listings.
- Users shall be able to browse listings and apply filters.
- Users shall be able to open listing details.
- Users shall be able to message listing owners.

Functional Requirements — Continued

- Users shall be able to add and remove favorites.
- Users shall be able to manage their own listings.
- Users shall be able to initiate a Buy Now Deal.
- Buyer and seller shall be able to confirm deal progress.
- Users shall be able to initiate and respond to exchange requests.
- Listings shall have controlled status values such as Available, Sold, Exchanged, and Removed.
- The system shall store optional latitude and longitude values for nearby discovery.
- Users shall be able to view and update profile information.
- Completed deals may be reviewed.

Security Requirements

- Passwords must not be stored as plain text.
- Protected API calls must use an Authorization Bearer token.
- Ownership checks must prevent unauthorized listing changes.
- A user must not create a deal against their own listing.
- Validation must be performed on API input before database operations.

3.6 Software Engineering Paradigm

Development Model: Adapted Waterfall Model

The development of Campus Exchange follows a structured, phase-based approach similar to an adapted Waterfall model. The project is divided into clear stages, while practical feedback from testing and debugging can be used to refine implementation.

- Structured phase progression — requirements, design, coding, testing, and deployment are clearly identified.
- Iterative refinements — defects discovered during testing can be corrected before final deployment.
- Defined milestones — major modules are completed and verified in stages.
- Modular development — authentication, listings, messaging, deals, exchanges, and profile features are separated.

Development Approach

Requirement Analysis → System Design → Implementation → Testing → Deployment → Maintenance

This approach is appropriate for an academic full-stack application because it provides clear documentation checkpoints while still allowing practical debugging during implementation.

3.7 Phases of Development

- 1. Requirement Analysis & System Study — identify student marketplace requirements and define the scope.
- 2. System Design — define modules, database relationships, API responsibilities, and UI navigation.
- 3. Implementation (Coding) — build React pages, FastAPI routers/services, SQLAlchemy models, and authentication.
- 4. Integration — connect frontend APIs, MySQL persistence, image handling, and browser location.
- 5. Testing & Debugging — verify individual features and end-to-end workflows.
- 6. Deployment — publish the frontend/backend to a hosting environment and configure production services.
- 7. Maintenance — improve usability, fix defects, and add future marketplace features.

Project Architecture

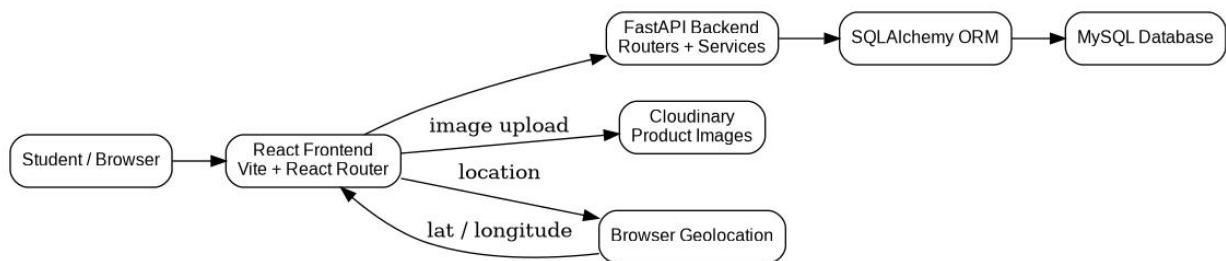


Fig. 3.1: High-level architecture of Campus Exchange.

3.8 Data Flow Diagram

A Data Flow Diagram (DFD) represents how information moves between external users, system processes, and data stores. In Campus Exchange, the student is the primary external entity and the application coordinates listing, messaging, transaction, and profile operations.

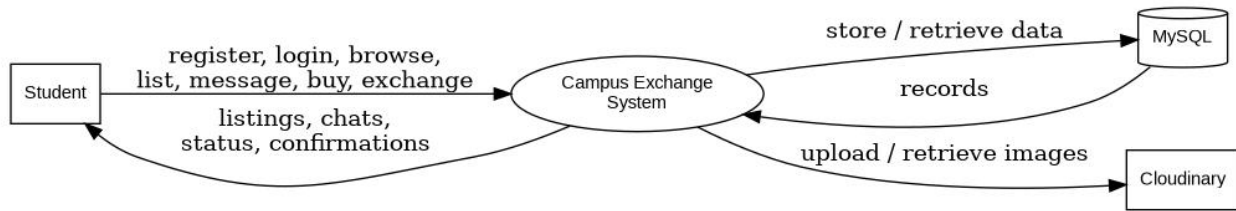


Fig. 3.2: Zero-level DFD of Campus Exchange.

The system receives user actions such as registration, login, browsing, listing creation, messaging, buying, and exchange requests. Persistent records are stored in MySQL, while product images may be stored through Cloudinary. These actions are transformed into validated application operations and persistent records, while media can be handled through hosted image storage.

First-Level DFD

The first-level view decomposes the main Campus Exchange process into the major functional areas used by students.

- Authentication — registration, login, token validation, and current-user retrieval.
- Listing Management — create, read, update, and remove listings.
- Search & Discovery — browse categories, filters, favorites, and nearby sellers.
- Communication — listing-linked student messaging.
- Transactions — Deal and Exchange workflows.
- Profile & Reviews — student profile data and post-transaction feedback.

Student → Authentication → Listing/Discovery → Communication → Deal/Exchange → Database

Second-Level DFD — Transactions

The transaction workflow contains two principal paths: normal purchase and exchange.

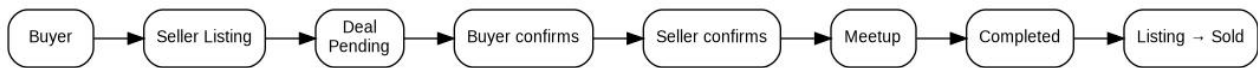


Fig. 3.3: Purchase/Deal process flow.

For exchanges, the requester selects an item to offer and an item to request. The receiver can accept or reject the request. A completed exchange changes the relevant transaction state and can be reflected in listing status.

Second-Level DFD — User Interaction

A user can browse items and then choose different actions depending on the listing.

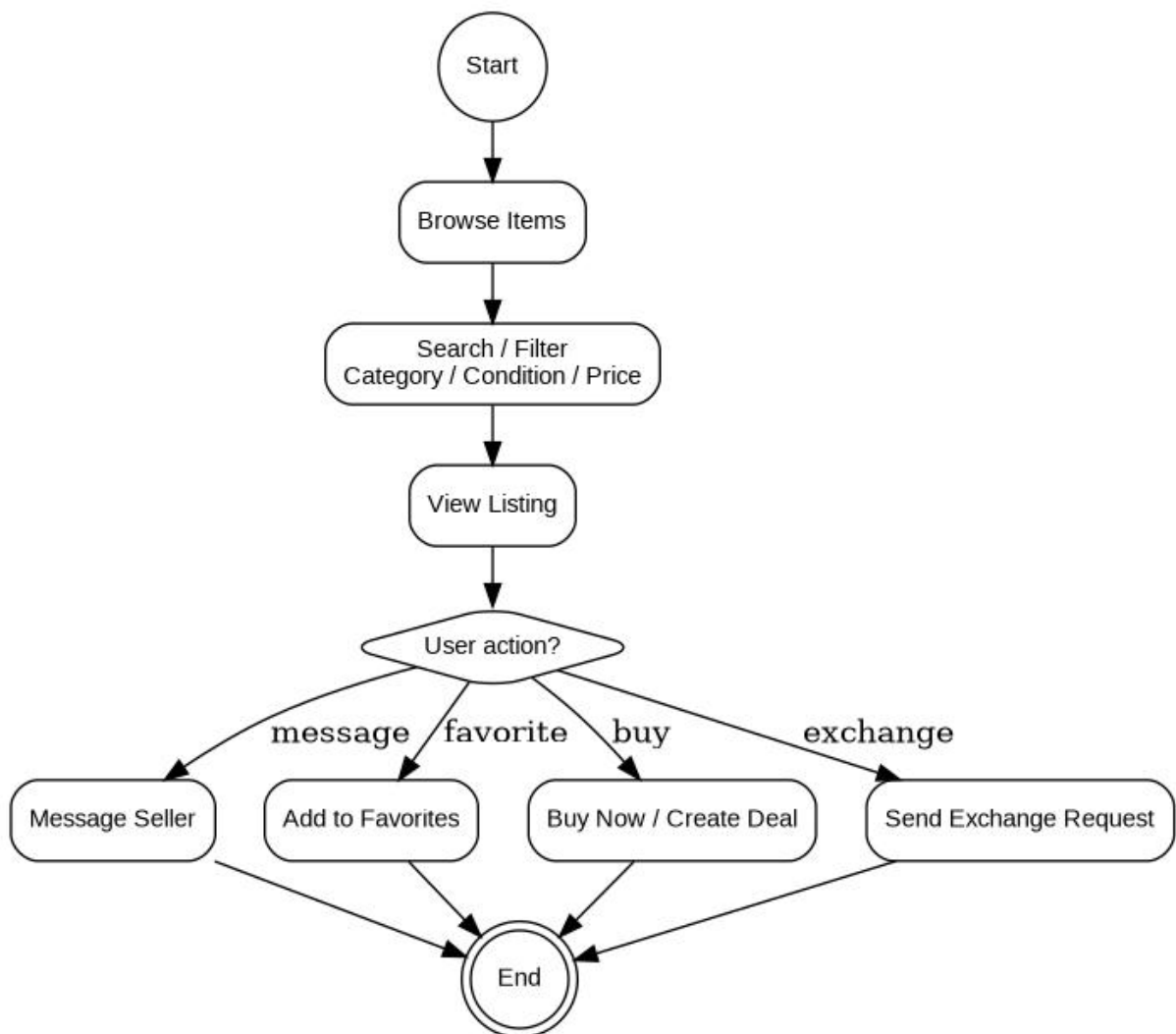


Fig. 3.4: User interaction flow.

The same listing can therefore lead to a message, favorite action, purchase workflow, or exchange request. This design keeps discovery independent from the final transaction choice.

Data Flow — Listing Creation

The listing creation process combines form validation, optional image upload, browser geolocation, API submission, and database persistence.

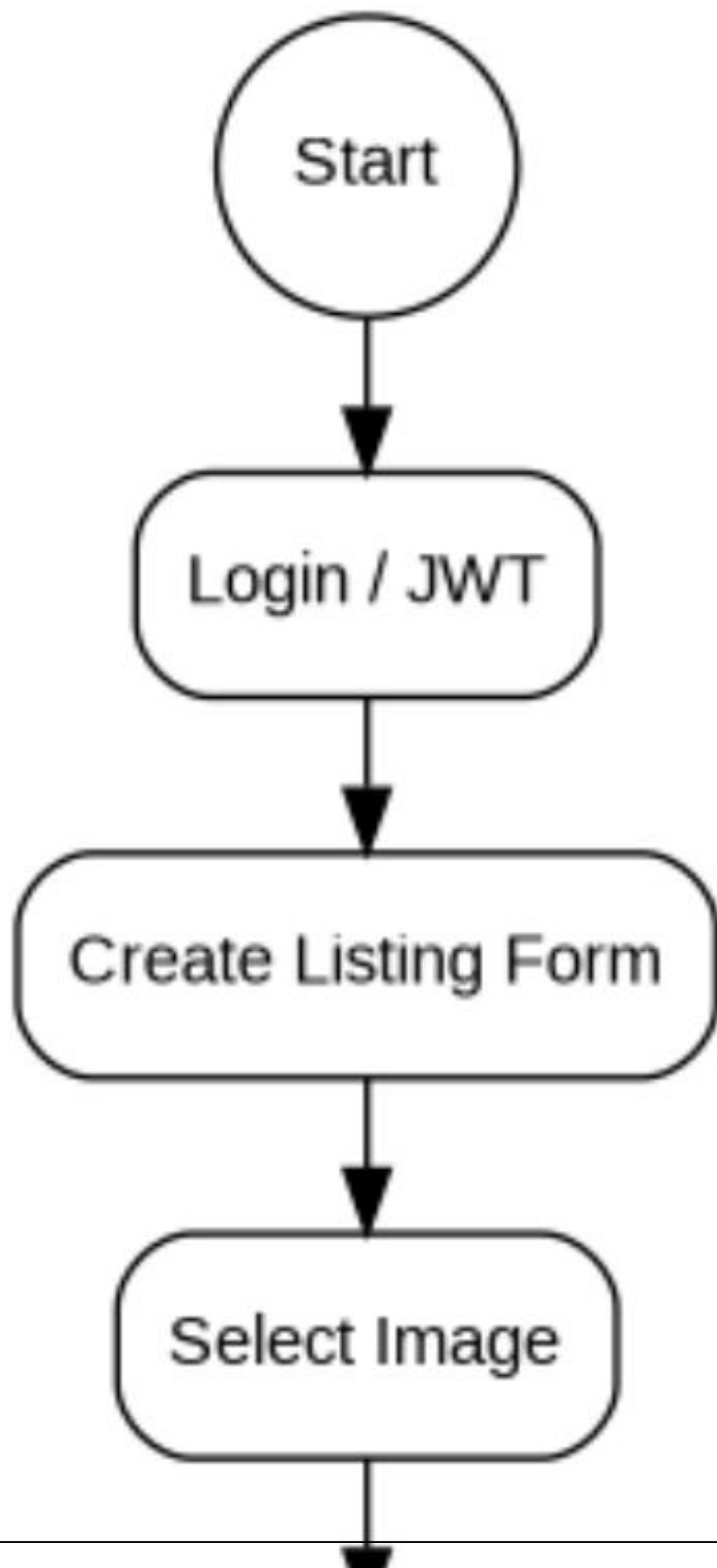


Fig. 3.5: Listing creation workflow.

The frontend collects title, description, price, category, condition, listing type, and optional location. The backend validates the request and stores the listing. Images can be represented by hosted image URLs.

System Data Flow — Messaging and Discovery

Messaging is linked to a seller and listing so that a conversation retains context about the product being discussed.

```
Example route context:  
/messages?receiver_id=<seller_id>&listing_id=<listing_id>
```

Nearby Sellers uses browser geolocation. The frontend can request the current latitude and longitude, then use the coordinates to retrieve nearby listings or sellers according to the selected search radius.

Favorites provide another discovery mechanism: a student can save a listing and later open it from the Favorites page without searching again.

ER Diagram

The Entity Relationship design organizes the major persistent entities of the application. Users create listings, listings belong to categories, listing images belong to listings, and transactional entities connect users and listings. It reflects the real relationships in the marketplace: users own listings, listings belong to categories, students save listings as favorites, and users participate in messages, deals, and exchanges.

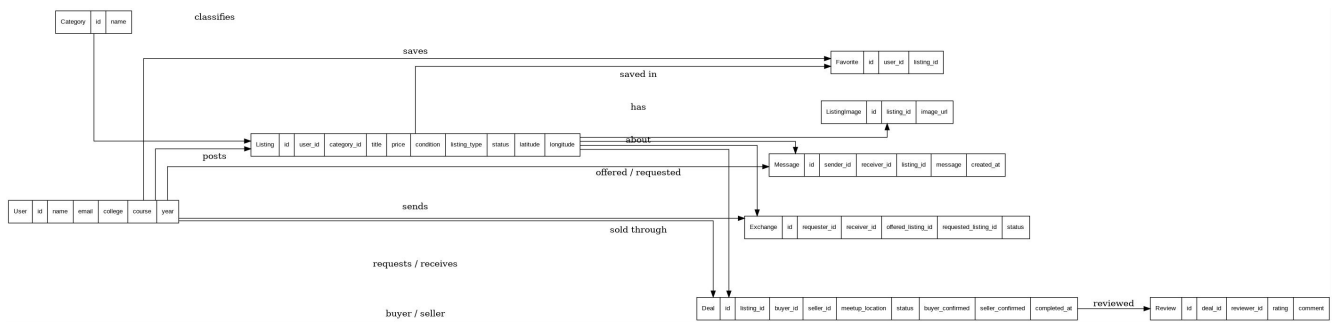


Fig. 3.6: Relationship between major Campus Exchange database entities.

4. System Design

4.1 Module

Campus Exchange is divided into functional modules so that each part of the application has a clear responsibility. This modular structure improves maintainability and makes it possible to extend individual features without redesigning the complete application.

- User Module — registration, login, profile, and account information.
- Listing Module — create, view, update, remove, and manage listings.
- Browse Module — category selection, search, filtering, and listing details.
- Favorites Module — save and remove preferred listings.
- Messaging Module — communicate with other students about listings.
- Deal Module — purchase workflow and buyer/seller confirmation.
- Exchange Module — request, accept/reject, and complete item exchanges.
- Nearby Sellers Module — location-based listing discovery.
- Review Module — feedback after completed deals.

4.2 Data Structure of All Modules

The database is organized around relational entities. SQLAlchemy models represent the application's database records and relationships. Primary keys identify individual records and relationships connect users, listings, categories, messages, favorites, deals, and exchanges.

Entity	Purpose	Important fields
User	Student account and profile	id, name, email, college, course, year
Category	Listing category	id, name
Listing	Product posted by a student	title, description, price, condition, type, status
ListingImage	Hosted listing image	listing_id, image_url
Favorite	Saved listing	user_id, listing_id
Message	Student conversation message	sender_id, receiver_id, listing_id, message
Exchange	Item exchange request	requester, receiver, offered/requested listings, status
Deal	Normal purchase transaction	buyer, seller, listing, meetup, confirmations, status
Review	Post-deal feedback	deal, reviewer, rating, comment

The relational structure separates reusable information from transaction-specific information, making the system easier to query and maintain.

Database Structure — Listings and Categories

The Listings entity stores the core marketplace data. The captured MySQL output shows fields including user_id, category_id, title, description, price, condition, listing_type, status, latitude, longitude, and created_at. The captured MySQL records demonstrate fields for ownership, category, title, description, price, condition, listing type, status, and optional geographic coordinates.

```
mysql> select * from listings;
```

id	user_id	category_id	title	description	price	condition	listing_type	status	latitude	longitude	created_at
1	1	1	Operating System Concepts	10th edition book in good condition	450.00	Good	Sell	Removed	NULL	NULL	2026-08-10 00:45:09
3	2	1	Python Programming	Python book in very good condition	350.00	Good	Exchange	Available	NULL	NULL	2026-08-19 14:36:54
4	1	1	Python Programming Book	Python book in excellent condition	400.00	Like New	Sell	Available	22.57260800	88.36390000	NULL
5	1	1	Python Programming Book	Python book in excellent condition	400.00	Like New	Sell	Available	22.57260800	88.36390000	NULL
6	1	1	Python Programming Book	Python book in excellent condition	350.00	Like New	Sell	Available	22.57260800	88.36390000	NULL
7	3	1	JWT Test Book Updated	Updated by the listing owner	250.00	Like New	Sell	Removed	19.24080800	73.13080800	NULL
8	5	3	casio scientific calculator	calculator to solve hard maths and it's 2month old	650.00	Like New	Sell	Available	NULL	NULL	NULL
9	5	3	HP Calculator	Scientific calculator in good condition	499.00	Good	Sell	Available	NULL	NULL	NULL
10	7	4	dissection box	proper tool for lab	300.00	Used	Sell	Removed	NULL	NULL	NULL
11	8	2	dsa notes	dsa notes	20.00	Good	Sell	Available	NULL	NULL	NULL
12	7	5	geometry box	help in geometry	100.00	Good	Sell	Available	19.28154000	73.07395900	NULL
13	10	5	pencil	pencil	10.00	Good	Sell	Available	NULL	NULL	NULL
14	11	1	python programming book	using this book you can learn python very fast	650.00	Like New	Sell	Available	19.28155500	73.07392300	NULL
15	12	1	DSA book	DSA learning book	500.00	Good	Sell	Available	19.28155128	73.07391188	NULL

```
14 rows in set (0.00 sec)
```

```
mysql>
```

Fig. 4.1: MySQL listing records used by Campus Exchange.

The listing data supports both normal sales and exchanges through the listing_type field, while status represents the current availability state.

Database Structure — Listing Images and Favorites

Listing images are stored separately from the main listing record so that a listing can support one or more hosted images without duplicating product data.

Favorites are represented by a relationship between a user and a listing. This allows the same listing to be saved by multiple students while keeping each student's saved-items state independent.

```
Listing 1 —— N ListingImage  
User 1 —— N Favorite N —— 1 Listing
```

This separation also keeps the listing table focused on marketplace attributes while media and personalization remain independent concerns.

Database Structure — Deals and Messages

Deals and messages represent interaction records that are separate from the basic listing record. This separation keeps product information independent from communication and transaction history, allowing the same listing to participate in different interactions over time.

Deal Structure

- listing_id identifies the item being purchased.
- buyer_id and seller_id identify the transaction participants.
- meetup_location stores the agreed meeting place.
- status represents Pending, Meetup, Completed, or Cancelled.
- buyer_confirmed and seller_confirmed track separate confirmation states.
- completed_at records the completion time.

Message Structure

- sender_id identifies the student sending the message.
- receiver_id identifies the student receiving the message.
- listing_id connects the conversation to a product.
- message stores the conversation content.
- created_at stores the message timestamp.

Database Screenshots

```
mysql> desc deals;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
listing_id	int	NO	MUL	NULL	
buyer_id	int	NO	MUL	NULL	
seller_id	int	NO	MUL	NULL	
meetup_location	varchar(300)	YES		NULL	
status	enum('Pending', 'Meetup', 'Completed', 'Cancelled')	YES		Pending	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
completed_at	timestamp	YES		NULL	
buyer_confirmed	tinyint(1)	NO		0	
seller_confirmed	tinyint(1)	NO		0	

10 rows in set (0.01 sec)

Fig. 4.2: MySQL DESCRIBE output for the Deal details table.

```
mysql> select * from messages;
```

id	sender_id	receiver_id	listing_id	message	created_at
1	1	2	6	Is this book still available?	NULL
4	1	2	6		NULL
6	1	2	6	Testing timestamp	2026-08-30 01:51:15
7	3	2	3	Hello, I am interested in this listing.	2026-08-30 16:47:43
9	5	2	3	Is this book still available?	2026-08-31 01:17:27
10	5	2	3	its available	2026-08-31 14:12:12
11	5	1	4	hiiii	2026-08-31 14:12:37
12	5	2	3	hiiii	2026-08-31 14:23:26
13	7	1	4	its available now bro	2026-08-31 14:31:15
14	7	1	4	yess bro its available	2026-08-31 14:35:28
15	7	1	4	okay i want to buy	2026-08-31 14:35:53
16	5	1	4	hiiii	2026-08-31 20:02:36
17	7	1	4	hii	2026-08-31 20:04:49
18	7	1	4	hello	2026-08-31 20:05:06
19	8	7	11	hii	2026-09-01 01:59:50
20	7	8	11	hii	
2026-09-01 02:00:12					
21	8	7	11	hiiii	2026-09-01 02:00:33
22	8	7	11	hello	2026-09-01 02:00:41
23	10	10	13	hiiii i want to buy this	
2026-09-01 13:36:08					
24	10	10	13	okay	2026-09-01 13:36:25
25	10	10	13	hii	
2026-09-01 13:37:06					
26	7	8	11	hiiii	2026-09-01 16:16:09
27	8	7	11	hiiii	2026-09-01 16:16:23
28	5	5	8	hii	2026-09-02 12:35:43
29	5	5	8	hiiii	2026-09-02 12:36:15
30	5	7	12	hiiii i want to buy your geometry box	2026-09-02 12:37:06
31	5	7	12	hiiii	
2026-09-02 12:37:43					
32	5	7	12	hiiii can you hear me	2026-09-02 12:42:21
33	5	5	8	hello	2026-09-02 12:43:20
34	8	5	8	i want to buy this	
2026-09-02 12:47:34					
35	5	8	8	okay	2026-09-02 12:47:59
36	12	11	14	hiiii	2026-09-02 12:53:46
37	11	12	14	hello you want to buy	2026-09-02 12:54:11
38	11	12	14	hiiii when we meet	2026-09-02 12:55:39

34 rows in set (0.00 sec)

Fig. 4.3: MySQL message records.

These screenshots demonstrate that transaction and communication data are persisted as structured database records.

Relationship Between Tables (Class Diagram)

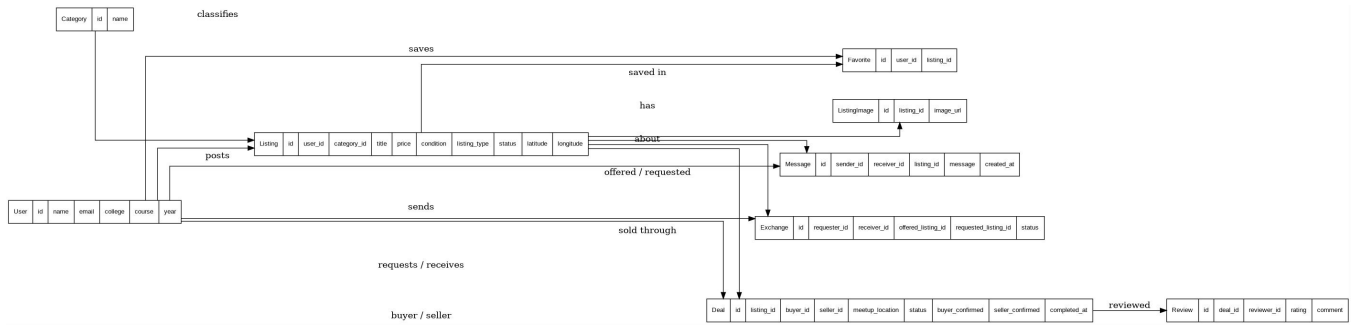


Fig. 4.4: Simplified class/entity relationship diagram for the application.

The diagram emphasizes the most important application relationships rather than implementation-specific framework tables.

4.3 Procedural Design

4.3.1 User Panel Design

In the user panel, a student can browse products, filter results, open a listing, and then choose a marketplace action. The design keeps the main navigation visible so that students can move between Dashboard, Browse Items, My Listings, Nearby Sellers, Exchanges, Favorites, Messages, Deals, and Profile. The navigation also provides direct access to My Listings, Nearby Sellers, Exchanges, Favorites, Messages, Deals, and Profile, keeping the main workflows accessible.

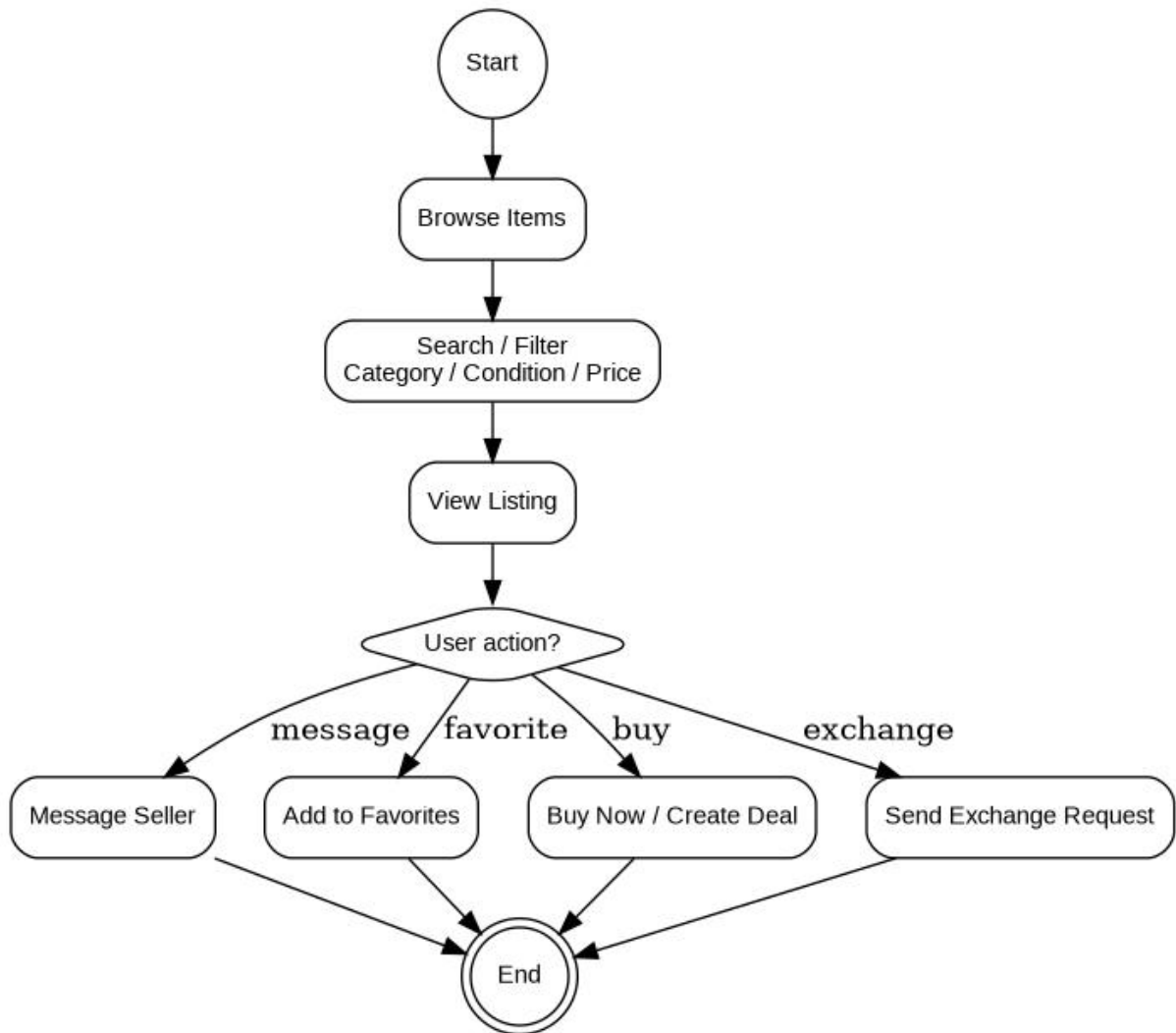


Fig. 4.5: User panel flowchart.

4.3.2 Create Listing Design

The create-listing process collects product information from the student and prepares it for API submission. The form supports category selection, condition, listing type, price, description, product image, and optional location.

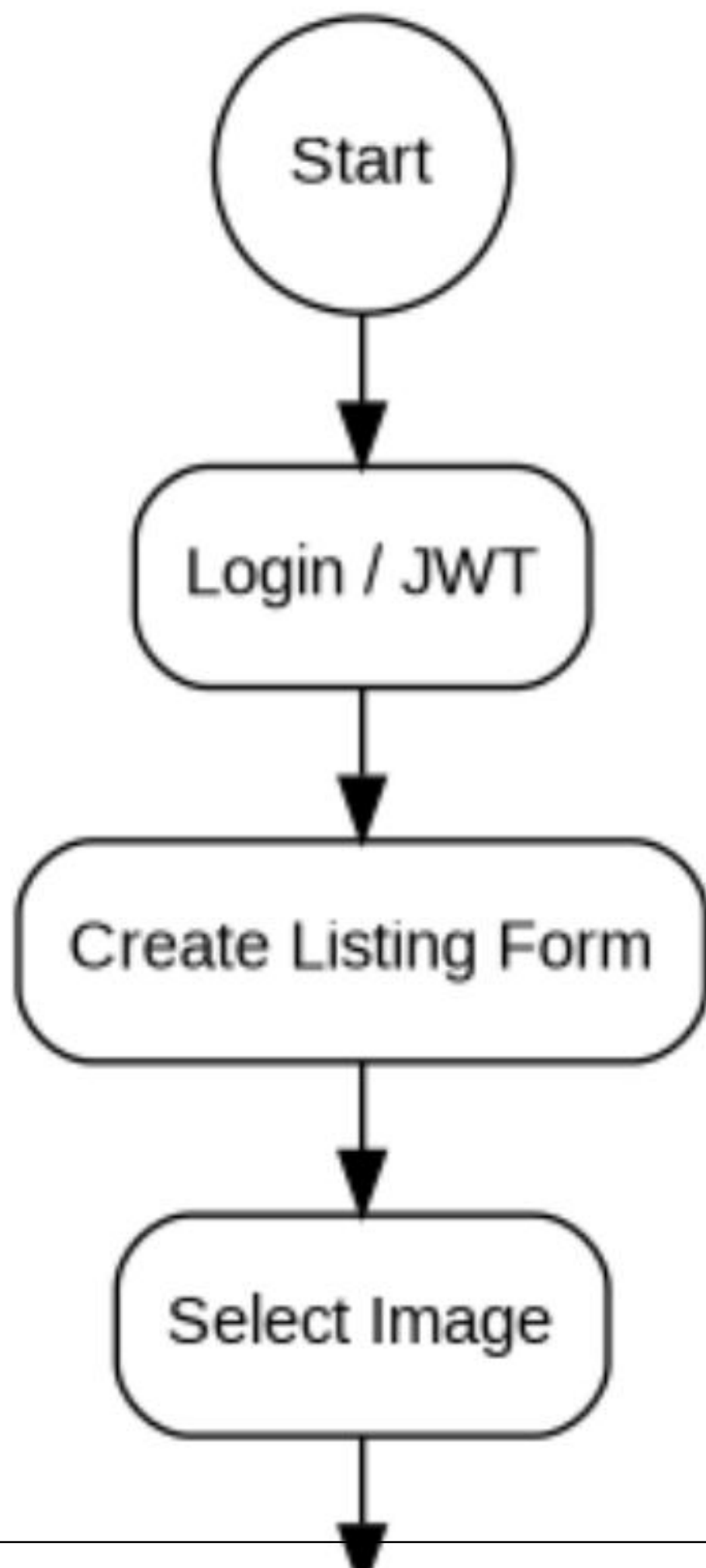


Fig. 4.6: Create listing flowchart.

The frontend also validates the selected image and can detect the student's current location through the browser Geolocation API.

4.3.3 Deal and Exchange Design

A normal purchase begins when a buyer selects Buy Now. The application creates a Deal and allows the participants to communicate and confirm the transaction. The separate confirmation fields make it possible to track buyer and seller confirmation independently. The Deal record separates buyer and seller confirmation, meeting information, status, and completion time so that transaction progress can be tracked explicitly.

Buy Now → Deal Created → Buyer/Seller Communication → Confirmation → Meetup → Completed

An Exchange follows a different path: the requester chooses a listing to offer, chooses the listing they want, sends the request, and waits for the receiver to accept or reject.

Offer Own Item + Request Item → Exchange Request → Accept/Reject → Complete

The two workflows are intentionally separated so that purchase transactions and item-for-item exchanges can have different business rules. This separation also makes the system easier to extend with additional transaction states or verification steps in the future.

SCREENSHOTS

Home Page / Dashboard

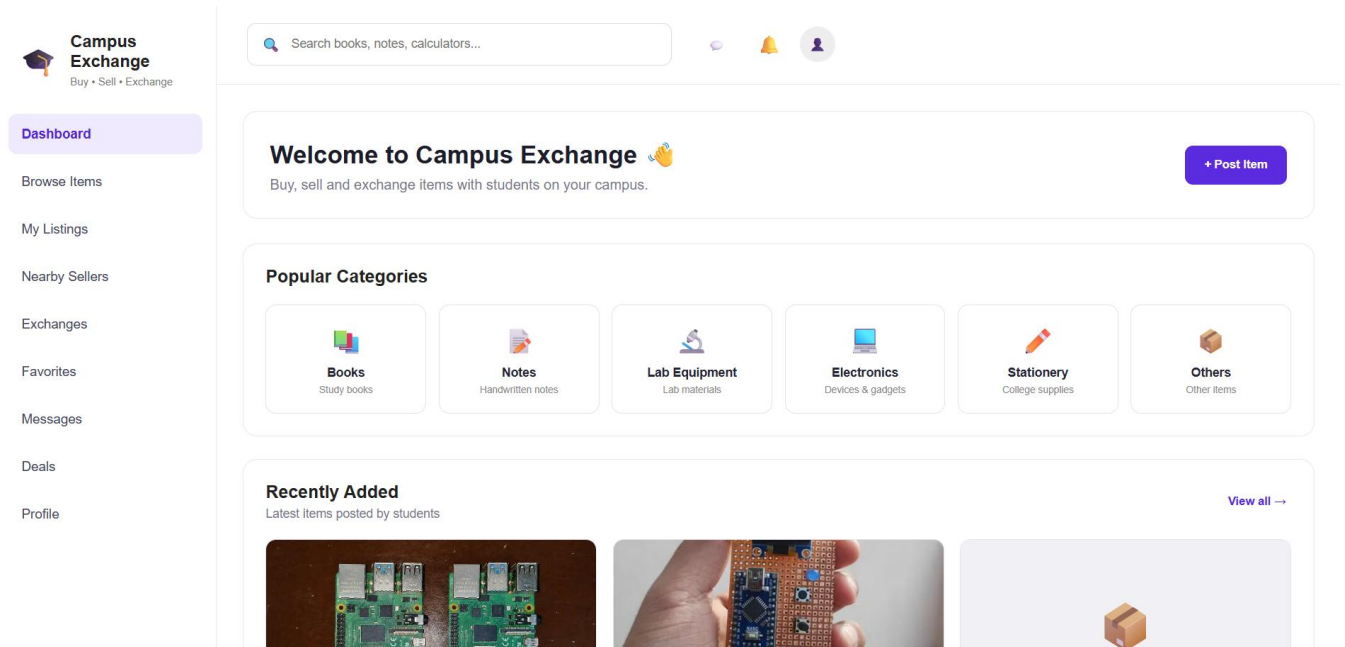


Fig. 5.1: Campus Exchange dashboard with popular categories and recently added items.

The dashboard provides a quick entry point to the marketplace and exposes the Post Item action.

Screenshots — Browse Items and My Listings

Browse Items

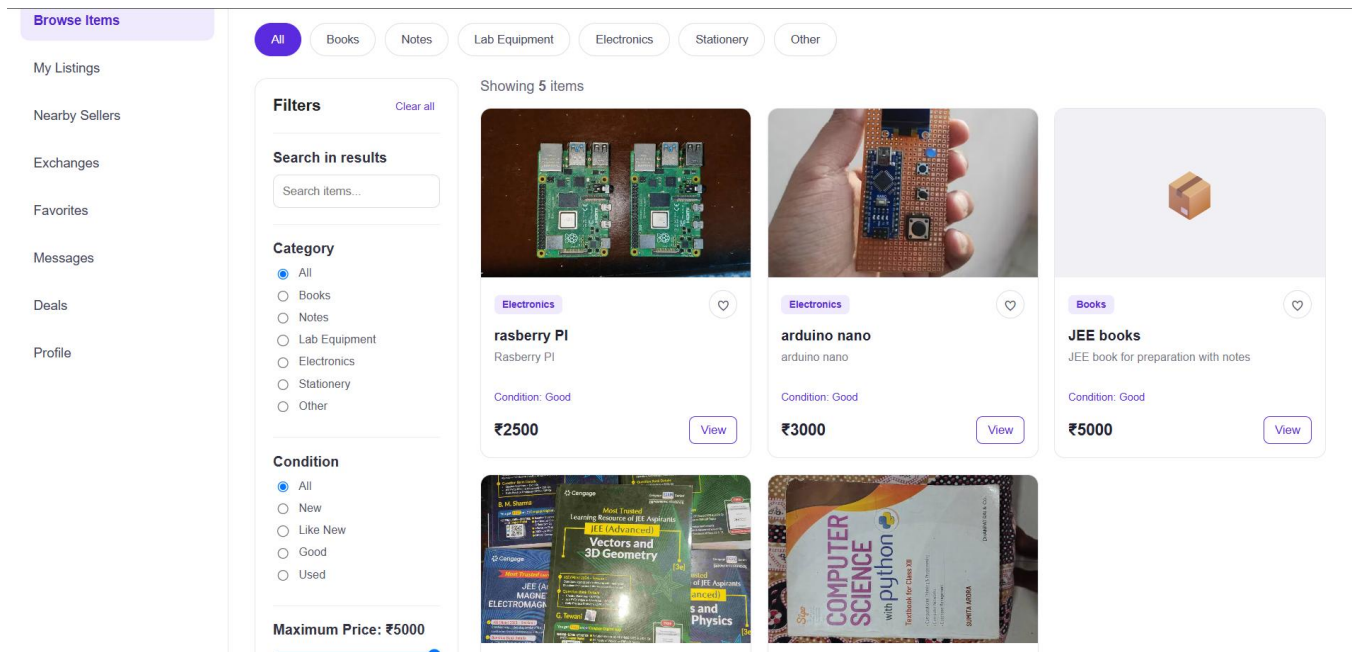


Fig. 5.2: Browse Items with category filters and listing cards.

My Listings

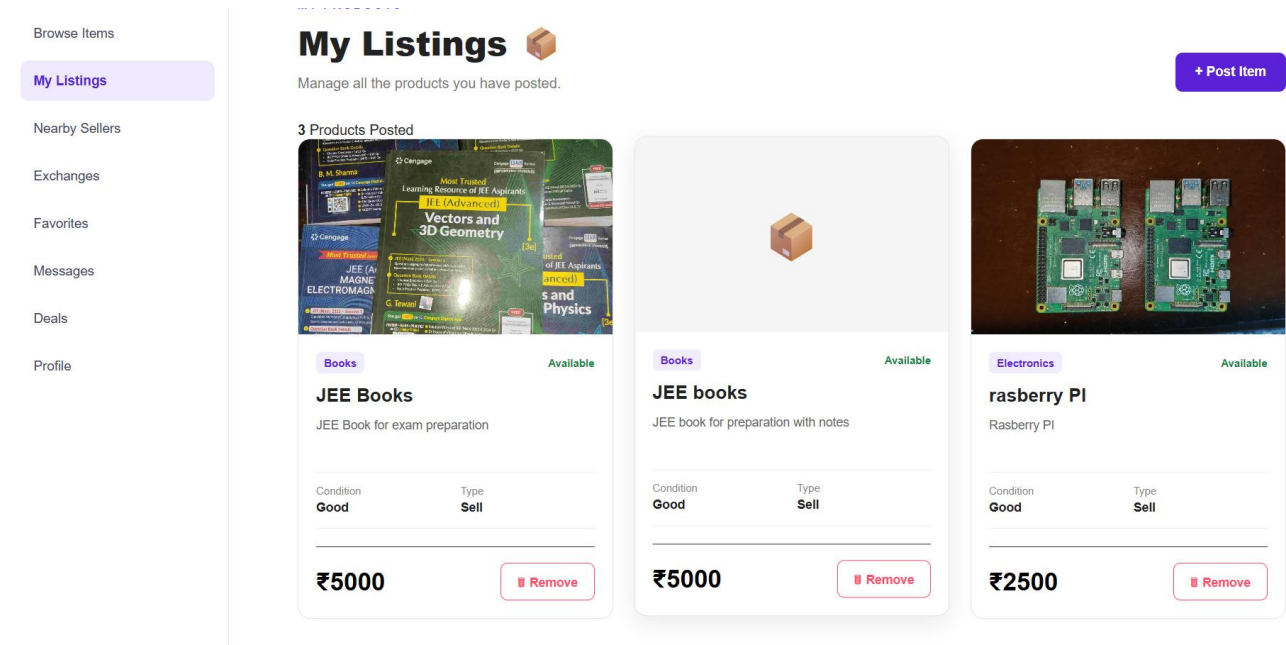


Fig. 5.3: My Listings page showing products posted by the logged-in user.

Screenshots — Exchanges and Profile

My Exchanges

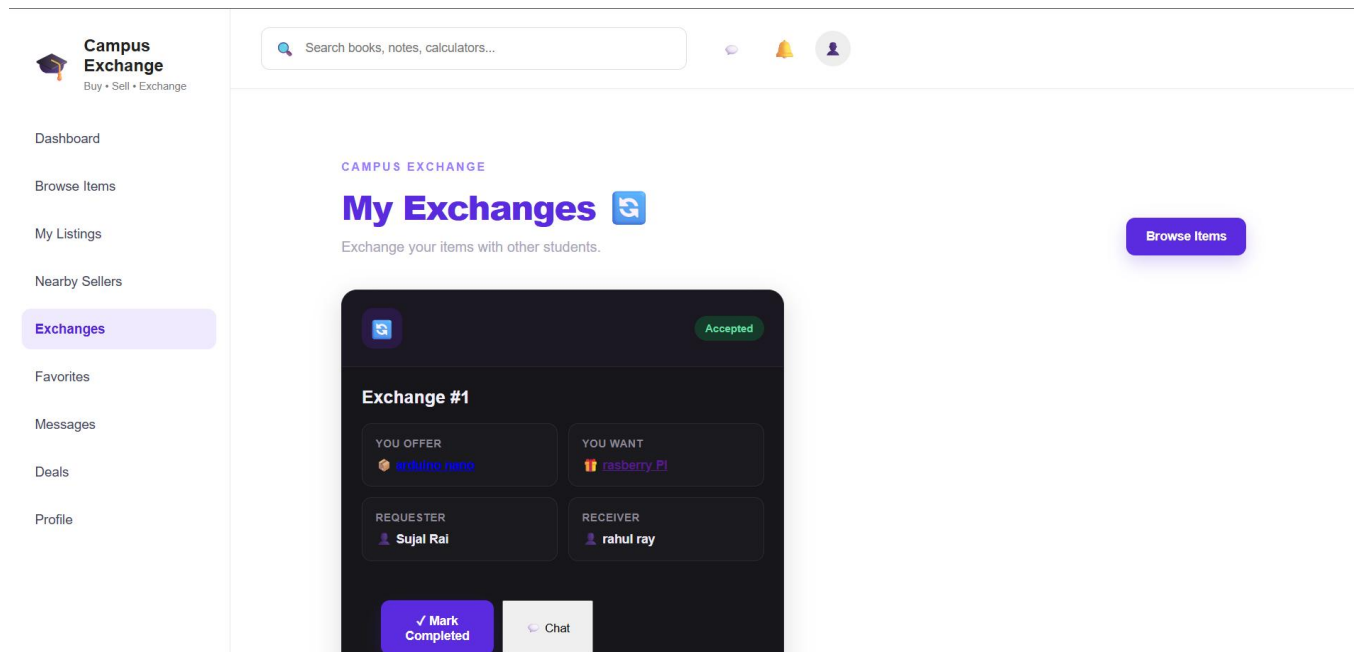


Fig. 5.4: Exchange request showing offered item, wanted item, requester, receiver, and accepted status.

My Profile

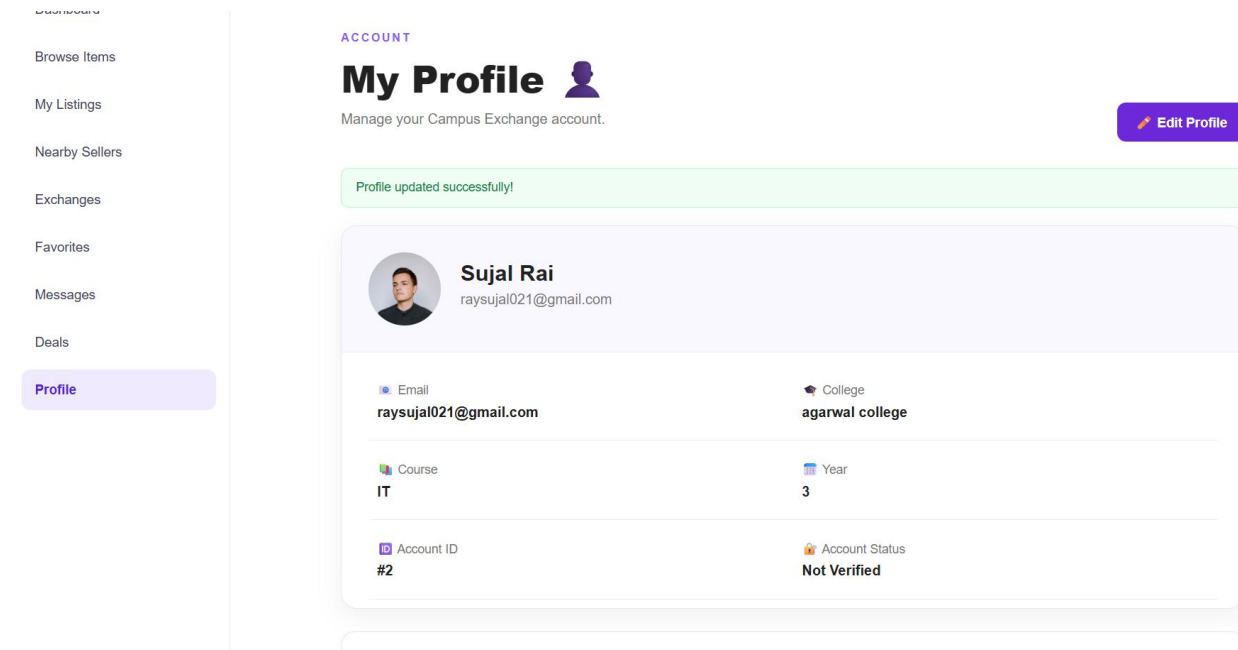


Fig. 5.5: Student profile page.

Screenshots — Messages and Nearby Sellers

Messages

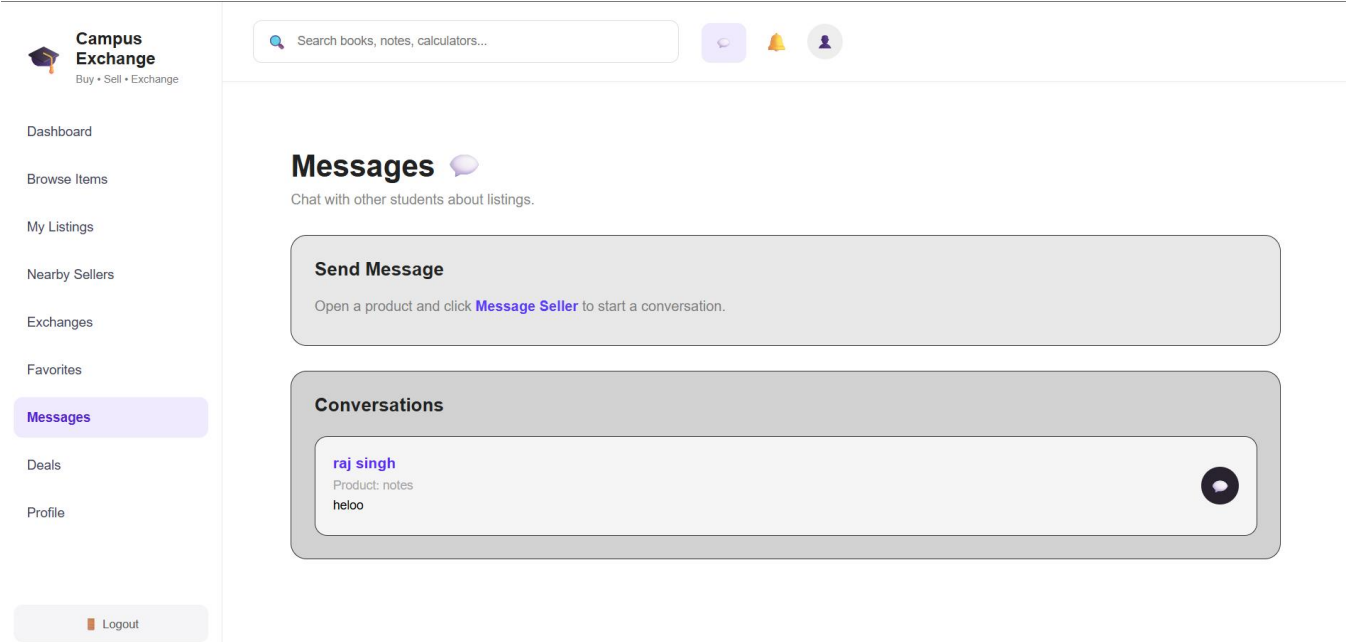


Fig. 5.6: Messages page showing listing-related conversations.

Nearby Sellers

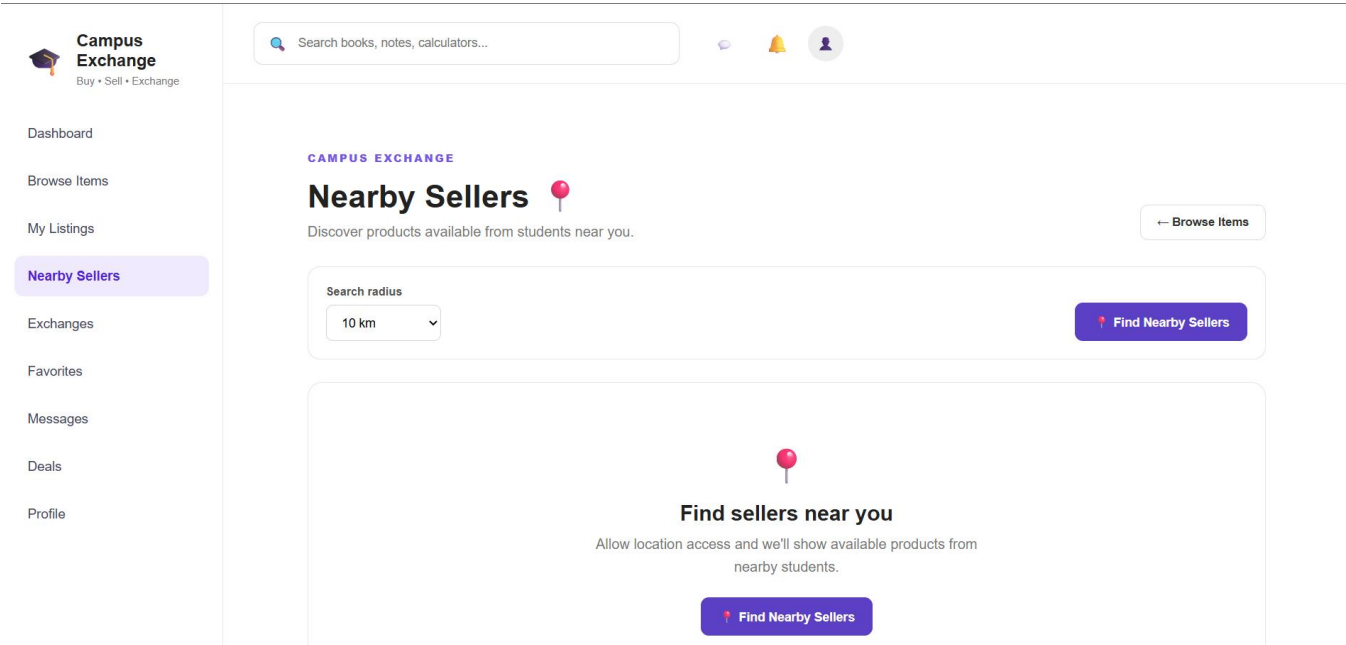


Fig. 5.7: Nearby Sellers page with configurable search radius.

Screenshots — Favorites and Database

Favorites

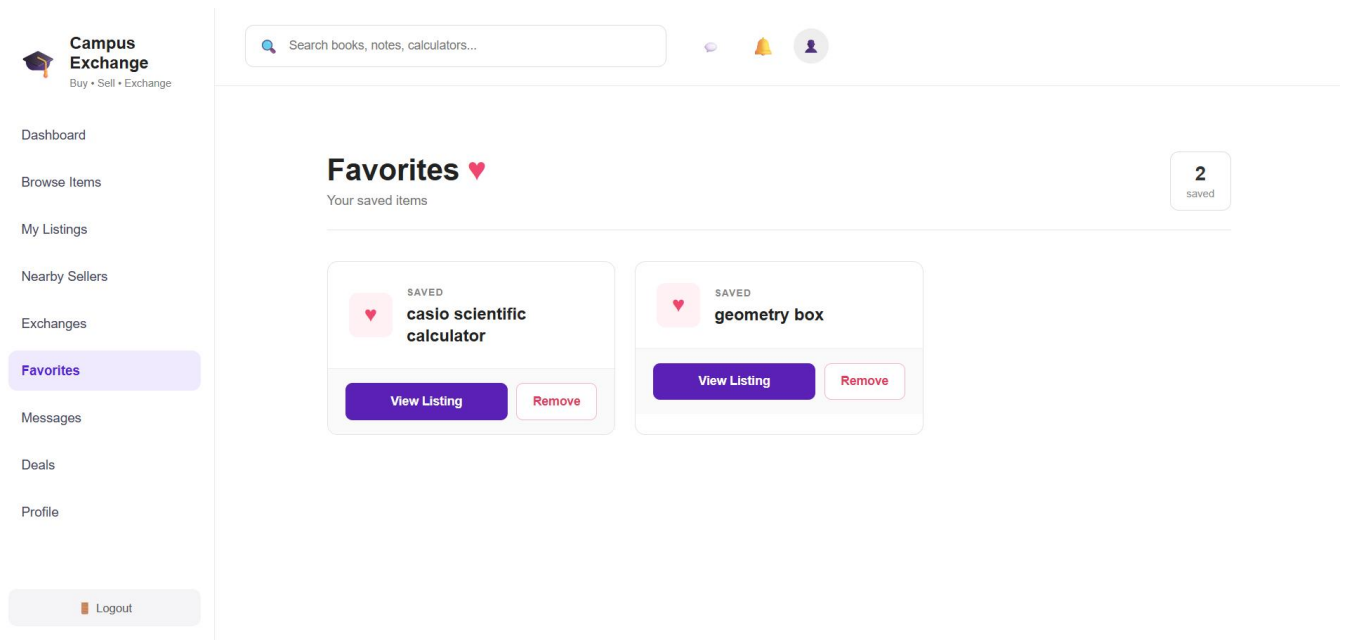


Fig. 5.8: Favorites page showing saved listings.

Database Listing Records

```
mysql> select * from listings;
```

id	user_id	category_id	title	description	price	condition	listing_type	status	latitude	longitude	created_at
1	1	1	Operating System Concepts	18th edition book in good condition	450.00	Good	Sell	Removed	NULL	NULL	2026-08-10 00:45:09
3	2	1	Python Programming	Python book in very good condition	350.00	Good	Exchange	Available	NULL	NULL	2026-08-19 14:36:54
4	1	1	Python Programming Book	Python book in excellent condition	400.00	Like New	Sell	Available	22.57260000	88.36390000	NULL
5	1	1	Python Programming Book	Python book in excellent condition	400.00	Like New	Sell	Available	22.57260000	88.36390000	NULL
6	1	1	Python Programming Book	Python book in excellent condition	350.00	Like New	Sell	Available	22.57260000	88.36390000	NULL
7	3	1	Self Test Book Updated	Updated by the listing owner	250.00	Like New	Sell	Removed	19.24080000	73.13080000	NULL
8	5	3	casio scientific calculator	calculator to solve hard maths and it's 2month old	650.00	Like New	Sell	Available	NULL	NULL	NULL
9	5	3	HP Calculator	Scientific calculator in good condition	499.00	Good	Sell	Available	NULL	NULL	NULL
10	7	4	dissection box	proper tool for lab	300.00	Used	Sell	Removed	NULL	NULL	NULL
11	8	2	notes	dsa notes	20.00	Good	Sell	Available	NULL	NULL	NULL
12	7	5	geometry box	help in geometry	100.00	Good	Sell	Available	19.28154000	73.07395900	NULL
13	10	5	pencil		10.00	Good	Sell	Available	NULL	NULL	NULL
14	11	1	python programming book	using this book you can learn python very fast	650.00	Like New	Sell	Available	19.28155504	73.07392249	NULL
15	12	1	DSA book	DSA learning book	500.00	Good	Sell	Available	19.28155128	73.07391188	NULL

14 rows in set (0.00 sec)

```
mysql>
```

Fig. 5.9: MySQL listing records.

Screenshots — Transaction and Message Data

Deal Table Structure

```
mysql> desc deals;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
listing_id	int	NO	MUL	NULL	
buyer_id	int	NO	MUL	NULL	
seller_id	int	NO	MUL	NULL	
meetup_location	varchar(300)	YES		NULL	
status	enum('Pending','Meetup','Completed','Cancelled')	YES		Pending	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
completed_at	timestamp	YES		NULL	
buyer_confirmed	tinyint(1)	NO		0	
seller_confirmed	tinyint(1)	NO		0	

10 rows in set (0.01 sec)

Fig. 5.10: MySQL DESCRIBE output for deal details.

Messages Table

```
mysql> select * from messages;
```

id	sender_id	receiver_id	listing_id	message	created_at
1	1	2	6	Is this book still available?	NULL
4	1	2	6		NULL
6	1	2	6	Testing timestamp	2026-08-30 01:51:15
7	3	2	3	Hello, I am interested in this listing.	2026-08-30 16:47:43
9	5	2	3	Is this book still available?	2026-08-31 01:17:27
10	5	2	3	its available	2026-08-31 14:12:12
11	5	1	4	hiiii	2026-08-31 14:12:37
12	5	2	3	hiiii	2026-08-31 14:23:26
13	7	1	4	its available now bro	2026-08-31 14:31:15
14	7	1	4	yess bro its available	2026-08-31 14:35:28
15	7	1	4	okay i want to buy	2026-08-31 14:35:53
16	5	1	4	hiiii	2026-08-31 20:02:36
17	7	1	4	hii	2026-08-31 20:04:49
18	7	1	4	hello	2026-08-31 20:05:06
19	8	7	11	hii	2026-09-01 01:59:50
20	7	8	11	hii	
2026-09-01 02:00:12					
21	8	7	11	hiiii	2026-09-01 02:00:33
22	8	7	11	heloo	2026-09-01 02:00:41
23	10	10	13	hiiii i want to buy this	
2026-09-01 13:36:08					
24	10	10	13	okay	2026-09-01 13:36:25
25	10	10	13	hii	
2026-09-01 13:37:06					
26	7	8	11	hiiii	2026-09-01 16:16:09
27	8	7	11	hiiii	2026-09-01 16:16:23
28	5	5	8	hii	2026-09-02 12:35:43
29	5	5	8	hiiii	2026-09-02 12:36:15
30	5	7	12	hiiii i want to buy your geometry box	2026-09-02 12:37:06
31	5	7	12	hiiii	
2026-09-02 12:37:43					
32	5	7	12	hiiii can you hear me	2026-09-02 12:42:21
33	5	5	8	hello	2026-09-02 12:43:20
34	8	5	8	i want to buy this	
2026-09-02 12:47:34					
35	5	8	8	okay	2026-09-02 12:47:59
36	12	11	14	hiiii	2026-09-02 12:53:46
37	11	12	14	hello you want to buy	2026-09-02 12:54:11
38	11	12	14	hiiii when we meet	2026-09-02 12:55:39

34 rows in set (0.00 sec)

Fig. 5.11: Stored message records and timestamps.

Screenshots — Combined Database Output

id	sender_id	receiver_id	listing_id	message	created_at
1	1	2	6	Is this book still available?	NULL
4	1	2	6	Testing timestamp	NULL
6	1	2	6	Hello, I am interested in this listing.	2026-08-30 01:51:15
7	3	2	3	Is this book still available?	2026-08-30 16:47:43
9	5	2	3	Is this book still available?	2026-08-31 01:17:27
10	5	2	3	its available	2026-08-31 14:12:12
11	5	1	4	hiiii	2026-08-31 14:12:37
12	5	2	3	hiiii	2026-08-31 14:23:26
13	7	1	4	its available now bro	2026-08-31 14:31:15
14	7	1	4	yess bro its available	2026-08-31 14:35:28
15	7	1	4	okay i want to buy	2026-08-31 14:35:53
16	5	1	4	hiiii	2026-08-31 20:02:36
17	7	1	4	hii	2026-08-31 20:04:49
18	7	1	4	hello	2026-08-31 20:05:06
19	8	7	11	hii	2026-09-01 01:59:50
20	7	8	11	hii	
2026-09-01 02:00:12					
21	8	7	11	hiiii	2026-09-01 02:00:33
22	8	7	11	heloo	2026-09-01 02:00:41
23	10	10	13	hiiii i want to buy this	
2026-09-01 13:36:08					
24	10	10	13	okay	2026-09-01 13:36:25
25	10	10	13	hii	
2026-09-01 13:37:06					
26	7	8	11	hiiii	2026-09-01 16:16:09
27	8	7	11	hiiii	2026-09-01 16:16:23
28	5	5	8	hii	2026-09-02 12:35:43
29	5	5	8	hiiii	2026-09-02 12:36:15
30	5	7	12	hiiii i want to buy your geometry box	2026-09-02 12:37:06
31	5	7	12	hiiii	
2026-09-02 12:37:43					
32	5	7	12	hiiii can you hear me	2026-09-02 12:42:21
33	5	5	8	hello	2026-09-02 12:43:20
34	8	5	8	i want to buy this	
2026-09-02 12:47:34					
35	5	8	8	okay	2026-09-02 12:47:59
36	12	11	14	hiiii	2026-09-02 12:53:46
37	11	12	14	hello you want to buy	2026-09-02 12:54:11
38	11	12	14	hiiii when we meet	2026-09-02 12:55:39

Fig. 5.12: Combined MySQL output showing listing and message records.

The captured output demonstrates that marketplace and communication information is stored persistently and can be queried directly from MySQL during development and verification.

Coding

React — Create Listing State

```
const [formData, setFormData] = useState({
  title: "",
  description: "",
  price: "",
  category_id: "",
  condition: "Good",
  listing_type: "Sell",
  latitude: "",
  longitude: ""
});
```

React — Automatic Location Detection

```
navigator.geolocation.getCurrentPosition((position) => {
  const latitude = position.coords.latitude;
  const longitude = position.coords.longitude;

  setFormData(previous => ({
    ...previous,
    latitude,
    longitude
  }));
});
```

The listing form uses React state to collect product data and the browser Geolocation API to populate optional coordinates.

Coding — Image and API Handling

Image Validation / Upload Preparation

```
const file = e.target.files?.[0];

if (!file) {
  setSelectedImage(null);
  setImagePreview("");
  return;
}

if (!file.type.startsWith("image/")) {
  setError("Please select a valid image file.");
  return;
}
```

API Request Pattern

```
const token = localStorage.getItem("token");

await axios.get("/deals/", {
  headers: {
    Authorization: `Bearer ${token}`
  }
});
```

The frontend stores the JWT access token in browser `localStorage` and sends it in the `Authorization` header for protected API calls.

Coding — Deal Workflow

Fetching Current User

```
const response = await axios.get(
  "http://127.0.0.1:8000/auth/me",
  {
    headers: {
      Authorization: `Bearer ${token}`
    }
  }
);
```

Refreshing Deal State

```
useEffect(() => {
  if (!currentUser) return;

  const interval = setInterval(() => {
    fetchDeals();
  }, 3000);

  return () => clearInterval(interval);
}, [currentUser]);
```

The deal page refreshes deal state periodically so that confirmation changes made in another browser session can become visible without requiring a manual refresh.

Coding — Backend Organization

```
backend/  
└─ app/  
    └─ models/  
    └─ schemas/  
    └─ services/  
    └─ routers/  
    └─ database/  
    └─ auth.py  
    └─ main.py
```

Representative API Design

```
GET    /users/  
GET    /users/{user_id}  
POST   /users/  
  
GET    /listings/  
GET    /listings/my  
GET    /listings/{listing_id}  
POST   /listings/  
PUT    /listings/{listing_id}  
DELETE /listings/{listing_id}
```

Routers expose feature-specific endpoints while service modules contain business and database logic. Pydantic schemas validate input and shape API responses.

Coding — Exchange and Deal APIs

```
GET    /exchanges/  
GET    /exchanges/{exchange_id}  
POST   /exchanges/  
PUT    /exchanges/{exchange_id}  
DELETE /exchanges/{exchange_id}
```

```
GET    /deals/  
GET    /deals/{deal_id}  
POST   /deals/  
PUT    /deals/{deal_id}  
DELETE /deals/{deal_id}
```

Exchange and Deal endpoints are protected where the operation depends on the current user. The backend uses ownership and participant checks to prevent unauthorized changes.

Coding — Database and Security

Authentication

```
Authorization: Bearer <JWT_ACCESS_TOKEN>
```

JWT authentication allows the backend to identify the current user on protected requests. Passwords are hashed before storage.

Database Relationships

```
User → Listings
Category → Listings
Listing → ListingImages
User → Favorites
User → Messages
Listing → Messages
User → Exchanges
Listing → Exchanges
User → Deals
Listing → Deals
```

SQLAlchemy provides an ORM layer so that these relationships can be represented as application models rather than raw SQL statements throughout the service layer.

Coding — Running the Project

Backend

```
cd backend
uvicorn app.main:app --reload
```

```
API:
http://127.0.0.1:8000
```

```
Swagger:
http://127.0.0.1:8000/docs
```

Frontend

```
cd frontend
npm install
npm run dev
```

```
Vite:
http://localhost:5173
```

The frontend and backend run as separate development processes and communicate through HTTP API requests.

Testing

Testing was performed at feature and workflow levels to verify that the application behaves according to the intended requirements. Key areas include authentication, navigation, listing creation, filtering, favorites, messaging, image handling, location capture, deals, and exchanges. Database records were also inspected during development to verify persistence.

ID	Test	Expected Result	Result
TC-01	Register user	Valid user account created	Pass
TC-02	Login	JWT token returned and protected pages accessible	Pass
TC-03	Create listing	Listing stored with category, price and condition	Pass
TC-04	Browse/filter	Matching listings displayed	Pass
TC-05	My Listings	Only logged-in user's listings shown	Pass
TC-06	Favorites	Listing can be saved and removed	Pass
TC-07	Messaging	Listing-linked message stored and displayed	Pass
TC-08	Buy Now	Deal created for another user's listing	Pass
TC-09	Deal confirmation	Buyer/seller confirmation state updated	Pass
TC-10	Exchange	Offer/request and status workflow works	Pass
TC-11	Nearby sellers	Location/radius workflow available	Pass
TC-12	Profile	Profile information displayed/updated	Pass

Testing includes unit-level checks of individual functions, module testing across related frontend/backend operations, integration testing between React and FastAPI, database verification, and system-level workflow testing.

Report

Application Result

Campus Exchange provides a functional student marketplace with separate navigation for discovery, personal listings, location-based discovery, exchanges, favorites, messages, deals, and profile management. The application brings these capabilities together with listing-linked messaging and transaction workflows, providing students with a single environment for campus resource sharing.

- The dashboard provides category shortcuts and recently added items.
- Browse Items presents listing cards with category, condition, price, and actions.
- My Listings provides ownership-focused listing management.
- Messaging provides listing-linked conversations between students.
- Favorites stores saved items for later access.
- Nearby Sellers provides a location-oriented discovery interface.
- Exchanges show offered/wanted items and participant roles.
- Deals provide purchase status and confirmation workflow.
- MySQL queries verify persistent listing, deal, and message data.

Observed Development Data

The provided development screenshots show a MySQL listings query returning 14 rows and a messages query returning 34 rows at the time of capture. These are development-state observations rather than production capacity measurements.

Deployment

The frontend has been tested in a hosted browser environment and the project uses external services such as Render and Cloudinary during deployment/integration. Production configuration should keep secrets and service credentials outside source code. The deployment configuration demonstrates how the frontend, backend API, database, and media service can operate together outside the local development environment.

6. Future Scope

- Campus email verification for stronger student identity assurance.
- Push notifications for messages, deals, and exchange requests.
- Real-time chat using WebSockets instead of periodic refresh.
- Advanced location search with maps and distance sorting.
- Multiple product images per listing with improved media management.
- Recommendation system based on browsing and favorite history.
- Admin moderation and reporting tools.
- Fraud/spam detection and stronger marketplace safety controls.
- Mobile application for Android and iOS.
- Analytics dashboard for listings, transactions, and engagement.
- Multilingual interface support.
- Improved review and reputation system.

Conclusion

Campus Exchange offers a practical solution for student-to-student buying, selling, and exchanging of campus resources. The system combines marketplace discovery with communication and transaction workflows, while a relational MySQL database provides persistent storage. React and FastAPI separate presentation from backend business logic, making the system easier to maintain and extend. It demonstrates how full-stack web technologies can be combined to solve a focused campus problem while providing a foundation for future features such as verification, notifications, real-time communication, and stronger moderation.

The project demonstrates full-stack development concepts including authentication, API integration, relational database design, file/media handling, geolocation, state management, and transaction workflows.

BIBLIOGRAPHY

- FastAPI documentation and learning resources.
- React documentation and component-based UI resources.
- SQLAlchemy ORM documentation.
- MySQL reference documentation.
- Vite documentation.
- Pydantic documentation.
- JWT authentication concepts and Python security resources.

REFERENCES

- FastAPI — <https://fastapi.tiangolo.com/>
- React — <https://react.dev/>
- SQLAlchemy — <https://www.sqlalchemy.org/>
- MySQL — <https://www.mysql.com/>
- Vite — <https://vite.dev/>
- Pydantic — <https://docs.pydantic.dev/>
- Cloudinary — <https://cloudinary.com/>
- Render — <https://render.com/>

Reference report format: the supplied Online News Portal with Sentiment Analysis report uses the same overall sequence of title, acknowledgement, abstract, introduction, objective, system analysis, system design, screenshots, coding, testing, report, future scope, conclusion, bibliography, and references.

Project Live Link : <https://campus-exchange-frontend-3ef1.onrender.com>

Github repo link : <https://github.com/Sujalgithub021/Sprint-1>